

FACTORY ANALYTICS BACKEND FLOW

A Comprehensive Deep Dive into Architectural Layers and
the End-to-End Request Journey

SYSTEM ENTRY POINT

Initialization, Global Configuration, and App Bootstrap in index.js

INITIALIZATION PHASE



Environment

Loads dotenv variables (DATABASE_URL, PORT) and establishes Mongoose connection via `mongoose.connect()` before handling requests.



Middleware

Global registration of `express.json()`, `CORS`, and `body-parser` to ensure consistent request parsing and security headers.



Lifecycle

Mounts API routes under versioned prefixes (`/v1/api`), registers global Error Handlers, and initiates `app.listen()`.

REQUEST LIFECYCLE

Traversing the Stack: From HTTP Entry to Data Persistence

| THE REQUEST JOURNEY

1. Route

Express matches URL and HTTP method (e.g., POST).

3. Controller

Orchestrates the req/res flow to Services.

2. Middleware

JWT Auth and Joi Schema Validation executed.

4. Data Model

Mongoose persists data to the MongoDB cluster.



MIDDLEWARE ECOSYSTEM



Auth (JWT)

`authenticateJwt` reads headers, verifies Bearer tokens, and sets `req.user` or triggers 401 Unauthorized.



Validation

`validate(schema)` uses Joi to sanitize `req.body`, ensuring strict data types before hitting the logic layer.



Handlers

Consistent JSON responses via `responseHandler`. Global error handler captures `next(err)` calls.

| VALIDATION & ERROR HANDLING

Joi Schema Logic

Applied to POST/PUT routes to ensure data integrity:

- **abortEarly: false** - Reports all errors at once.
- **stripUnknown: true** - Removes unmodeled fields.
- **Sanitization** - Converts and trims inputs automatically.

Global Error Handler

Captures all thrown exceptions:

- **Code 11000** - Formats Mongo duplicate key errors.
- **Custom Classes** - BadRequest (400), NotFound (404).
- **Safety Net** - Catches uncaught/unhandled rejections.

LOGIC ORCHESTRATION



Controllers vs. Services

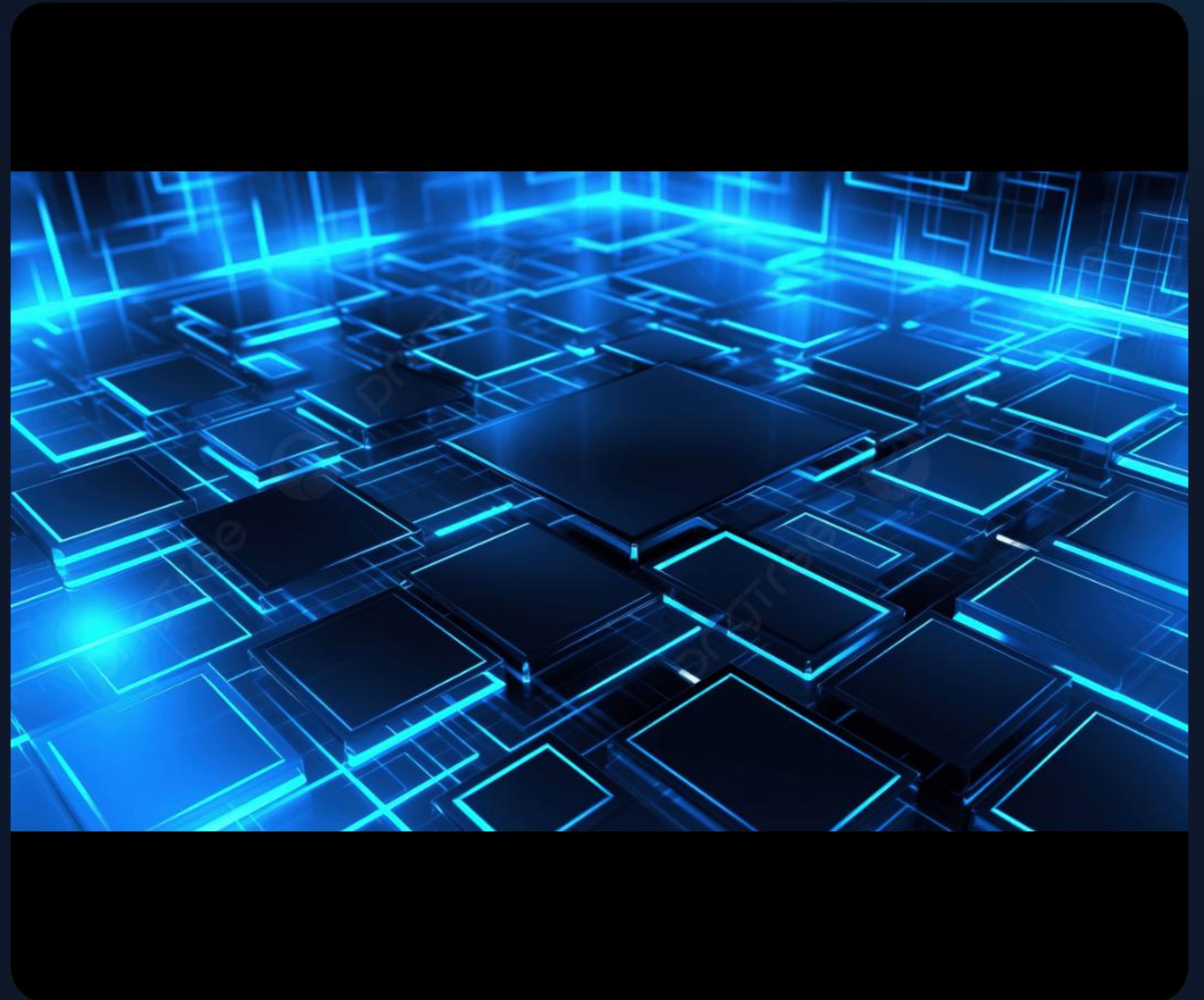
Controllers: Serve as traffic wardens. They read from req, call Services, and handle the HTTP response cycle. They contain zero database logic.

Services: The brain of the app. They house business calculations, aggregations, and multi-step operations using the DAL.

DATA PERSISTENCE LAYER

DAL & Mongoose Models

- **Generic DAL:** Model-agnostic data access exposing create, aggregate, and bulkWrite.
- **Service Decoupling:** Services pass Models to the DAL as arguments, avoiding direct Mongoose imports in logic.
- **Models:** Define the factory schema (Production, Order, AssemblyLine) and enforce Mongoose indexes.



FLOW EXAMPLE: PRODUCTION UPDATE

- ➔ **Request:** POST /v1/api/production-update arrives with JWT and JSON body.
- 👤 **Auth:** Middleware verifies token, sets `req.user` for ownership context.
- 🔹 **Validation:** Joi validates QC inputs, backlog counts, and line numbers.
- 🔧 **Service:** Business logic triggers `dal.create(ProductionUpdates, body)` .
- ✅ **Response:** Controller sends 200 OK with the newly created document.

RESPONSIBILITY MATRIX

Layer	Primary Responsibility	Focus Area
Routes	HTTP Pathing & Security Policy	Interface
Controllers	Request Mapping & Response Formatting	Orchestration
Services	Calculations, Logic, & DAL Calls	Business
DAL	Generic CRUD & Data Aggregation	Storage
Models	Schema Definition & Indexing	Structure

Questions?

Achieving Scalable Backend Architecture through Strict Separation of Concerns

FACTORY ANALYTICS BACKEND | TECHNICAL DESIGN REVIEW

IMAGE SOURCES



https://img.freepik.com/premium-photo/closeup-programming-code-screen-with-blurred-office-background-modern-software-development-coding-concept-with-warm-lighting_210545-21072.jpg?w=360

Source: www.freepik.com



https://png.pngtree.com/background/20231117/original/pngtree-d-rendering-of-blank-space-on-blue-technological-background-for-information-picture-image_6302296.jpg

Source: [pngtree.com](https://png.pngtree.com)
